7.1. Introduction to Deployment View

This section outlines the physical environment into which the Teaching Vacancies Python/Django system will be deployed. It describes the infrastructure components and how the software building blocks (Django applications, Celery workers, etc.) are mapped onto this infrastructure. This view is conceptual at this stage and will be refined as specific hosting decisions are finalized.

7.2. Target Environment (Assumed)

The system is expected to be deployed on a modern cloud platform. While the specific provider (e.g., AWS, Azure, Google Cloud, or a PaaS like GOV.UK PaaS) is subject to final DfE strategy and procurement, the architecture will be designed to be cloud-agnostic where possible.

A key assumption is the use of **containerization (Docker)** for packaging the Django application and Celery workers. This approach promotes consistency across development, testing, and production environments, simplifies dependency management, and facilitates scalable deployments, often orchestrated by systems like Kubernetes or platform-specific container services (e.g., AWS ECS, Azure App Service).

7.3. Key Infrastructure Components

The envisioned deployment environment will consist of the following key infrastructure components:

- **Web Server / Application Gateway / Load Balancer:**
- Acts as the entry point for all HTTP(S) traffic.
- Responsibilities: SSL termination, HTTP request routing, load balancing across application server instances, potentially serving static assets directly or via a CDN, and potentially providing a Web Application Firewall (WAF).
- Examples: Nginx, AWS Application Load Balancer (ALB), Azure Application Gateway.
- **Application Servers (Web Tier):**
- Hosts the Django web application.
- Multiple instances will run for scalability and high availability.
- Typically, a WSGI server like Gunicorn will be used to serve the Django application within each container/server instance.
- **Database Server (Data Tier):**

- A managed PostgreSQL service with the PostGIS extension enabled.
- Responsibilities: Persistent storage for all application data (users, organisations, vacancies, applications, etc.).
- Examples: AWS RDS for PostgreSQL, Azure Database for PostgreSQL.
- **Cache Server (Data Tier):**
- A managed Redis service.
- Responsibilities: Caching frequently accessed data to reduce database load, storing session information, and acting as a message broker for Celery.
- Examples: AWS ElastiCache for Redis, Azure Cache for Redis.
- **Background Task Processing Infrastructure (Task Processing Tier):**
- **Celery Workers:** Separate processes/containers running Celery worker instances. These consume tasks from the message queue (Redis) and execute them asynchronously (e.g., sending emails, data synchronization). Multiple worker instances will run for scalability.
- **Celery Beat Scheduler:** A single instance process responsible for scheduling periodic tasks (e.g., daily job alerts, data cleanup jobs).
- **Static & Media File Storage (Storage Tier):**
- Cloud-based object storage service.
- Responsibilities: Storing user-uploaded files (e.g., organisation logos, photos, application documents if any) and collected static assets (CSS, JavaScript, images served by the web server or CDN).
- Examples: AWS S3, Azure Blob Storage.
- **Logging Infrastructure:**
- A centralized logging solution to aggregate logs from all components (application servers, workers, database, etc.).
- Responsibilities: Log collection, storage, searching, and analysis for debugging and auditing.
- Examples: ELK Stack (Elasticsearch, Logstash, Kibana), AWS CloudWatch Logs, Azure Monitor Logs.
- **Monitoring & Alerting Infrastructure:**
- A system for monitoring application performance, server health, error rates, and other key metrics.
- Responsibilities: Metric collection, visualization (dashboards), and alerting based on predefined thresholds or anomalies.

- Examples: Prometheus/Grafana, Datadog, AWS CloudWatch, Azure Monitor.
- **Content Delivery Network (CDN) (Optional but Recommended):**
- For caching static assets closer to users, improving load times and reducing load on application servers.
- Examples: AWS CloudFront, Azure CDN, Cloudflare.

7.4. Deployment Diagram (Conceptual)

The following diagram illustrates a conceptual deployment of the Teaching Vacancies platform:

```
@startuml
!theme materia

cloud "Internet" {
  [User Browser]
  [External ATS System]
}

node "Cloud Platform (e.g., AWS, Azure, GOV.UK PaaS)" {
  node "VPC / Virtual Network" as VNet {
    node "Edge Services" as Edge {
      [CDN]
      [WAF]
      [Load Balancer (Public Facing)] as PubLB
    }

    node "Application Tier" as AppTier {
      collections "Django App Servers (Docker Containers with Gunicorn)" as DjangoServers
      [Django App Instance 1] <<application container>>
      [Django App Instance 2] <<application container>>
      [Django App Instance N] <<application container>>
      DjangoServers -- [Django App Instance 1]
      DjangoServers -- [Django App Instance 2]
      DjangoServers -- [Django App Instance N]
    }

    node "Task Processing Tier" as CeleryTier {
      collections "Celery Workers (Docker Containers)" as CeleryWorkers
      [Celery Worker 1] <<application container>>
      [Celery Worker 2] <<application container>>
      [Celery Worker M] <<application container>>
```

```
      CeleryWorkers -- [Celery Worker 1]
      CeleryWorkers -- [Celery Worker 2]
      CeleryWorkers -- [Celery Worker M]

      [Celery Beat Scheduler (Docker Container)] <<application container>> as CeleryBeat
    }

    node "Managed Data Services" as DataServices {
      database "PostgreSQL Server (with PostGIS)" as DBService
      node "Redis Service (Cache & Broker)" as RedisService
    }

    node "Managed Storage Services" as StorageServices {
      [Object Storage (S3 / Azure Blob)] <<storage>> as S3
    }
  }
}

package "External SaaS & GOV.UK Services" {
  [GOV.UK Notify API]
  [DfE Sign-In (OAuth/OIDC)]
  [GOV.UK One Login (OIDC)]
  [GIAS API/Data Feed]
  [DWP Find a Job API]
  [ONS Data API/Feed]
  [Central Logging Service]
  [Central Monitoring Service]
  [BigQuery / Data Warehouse]
}


UserBrowser --> PubLB
ExternalATSSystem --> PubLB : (ATS API Endpoint)
PubLB --> DjangoServers

DjangoServers --> DBService
DjangoServers --> RedisService
DjangoServers --> S3 : (Media Files)
DjangoServers --> [GOV.UK Notify API]
DjangoServers --> [DfE Sign-In (OAuth/OIDC)]
DjangoServers --> [GOV.UK One Login (OIDC)]
DjangoServers ..> [Central Logging Service]
DjangoServers ..> [Central Monitoring Service]

CeleryWorkers --> DBService
CeleryWorkers --> RedisService
```

```
CeleryWorkers --> S3 : (File processing if needed)
CeleryWorkers --> [GOV.UK Notify API]
CeleryWorkers --> [GIAS API/Data Feed]
CeleryWorkers --> [ONS Data API/Feed]
CeleryWorkers --> [DWP Find a Job API]
CeleryWorkers --> [BigQuery / Data Warehouse]
CeleryWorkers ..> [Central Logging Service]
CeleryWorkers ..> [Central Monitoring Service]

CeleryBeat --> RedisService : (Task Scheduling)
CeleryBeat ..> [Central Logging Service]
CeleryBeat ..> [Central Monitoring Service]

CDN ..> PubLB : (Origin Pull)
Edge ..> DjangoServers : (Routes Traffic)

@enduml
```

Key Interactions in Deployment Diagram:

- User traffic (from browsers and external ATS systems) hits the public-facing Load Balancer, potentially passing through a CDN and WAF.
- The Load Balancer distributes requests to the pool of Django Application Server instances.
- Django App Servers handle requests, interacting with:
- PostgreSQL/PostGIS database for primary data storage.
- Redis for caching and as a Celery message broker.
- Object Storage (S3/Azure Blob) for static assets and user-uploaded media.
- External identity providers (DfE Sign-In, GOV.UK One Login).
- GOV.UK Notify for some direct email sends.
- Centralized logging and monitoring services.
- Celery Workers (running in separate containers/processes) pick up tasks from Redis and interact with:
- PostgreSQL/PostGIS database.
- Redis (for task state).
- Object Storage (if tasks involve file processing).
- External services like GOV.UK Notify, GIAS, ONS, DWP Find a Job, and BigQuery.
- Centralized logging and monitoring services.

- Celery Beat Scheduler interacts with Redis to queue periodic tasks for the workers.
- Static assets are ideally served by the CDN or directly from Object Storage via the Load Balancer/Edge services to offload the application servers.

This conceptual deployment view provides a basis for more detailed infrastructure planning and CI/CD pipeline design.